



C# : accès aux fichiers

■ Différentes manières d'accès aux fichiers

- Classe **File**
 - ✓ Dispose de méthodes et propriétés statiques pour manipuler les fichiers
- Classe **FileInfo**
 - ✓ Nécessite d'instancier un objet pour manipuler un fichier
- Classe **My.Computer.FileSystem**
 - ✓ Existe à partir de la version 2005 de Visual Studio
- Classes du **stream** du framework
 - ✓ Classes de flux de lecture et écriture dans des fichiers
- Toutes ces classes sont disponibles dans **System.IO**



C# : accès aux fichiers

■ Classe File

- Dispose de méthodes statiques pour créer, lire, écrire, supprimer ou copier des fichiers
 - ✓ `File.Create(nom_fichier)` – `File.Delete(nom_fichier)`
 - ✓ `File.ReadAllText(fichier)` – `File.WriteAllLines(fichier)`
- Exemple :
 - *Lire la totalité d'un fichier texte*
`string montexte = File.ReadAllText("c:\montext.txt")`
 - *Lire et mettre dans un tableau les lignes d'un fichier texte*
`string meslignes = File.ReadAllLines("c:\montext.txt")`
 - *Tester si un fichier est en lecture seule*
`if (File.GetAttributes("c:\montext.txt") && FileAttributes.ReadOnly)`

...



C# : accès aux fichiers

■ Classe FileInfo

- Nécessite l'instanciation d'un objet et donne des informations sur un fichier particulier
 - ✓ `FileInfo Fi;`
`Fi = new FileInfo(nom_fichier);`
- Dispose de propriétés et méthodes pour manipuler un fichier
- Exemple : tester le Readonly d'un fichier
 - `string monfichier = "c:\\montext.txt";`
`FileInfo Fi = new FileInfo(monfichier);`
`if (Fi.Attributes && FileAttributes.ReadOnly) ...`



C# : accès aux fichiers

■ Les classes de stream du framework

- Sont utilisées pour lire et écrire dans un fichier en mode séquentiel
- En écriture on utilise la classe **StreamWriter**
 - ✓ Méthodes **Write**, **WriteLine** et **close** sont disponibles
 - ✓ Instanciation :
 - `StreamWiter Sw = new StreamWriter(nomfichier [,true]);`
 - `StreamWriter Sw = File.CreateText(nomfichier); // avec File`
 - `StreamWriter Sw = File.AppendText(nomfichier); //ajout`
- Exemple :
 - *`StreamWriter Sw = new StreamWriter("c:|montext.txt");
Sw.WriteLine("Salut");`*



C# : accès aux fichiers

■ Les classes de stream du framework

- En lecture on utilise la classe **StreamReader**
 - ✓ Méthodes **Read**, **ReadLine** et **ReadToEnd** sont disponibles
 - ✓ Instanciation :
 - `StreamReader Sr = new StreamReader(nomfichier);`
 - `StreamReader Sr = File.OpenText(nomfichier); // avec File`

■ Classe **My.Computer.FileSystem**

- Dispose des méthodes **CopyFile**, **DeleteFile**, **FileExists**, **RenameFile**, **MoveFile** et de la propriété **GetFiles** qui retourne une collection de fichiers dont **count** retourne le nombre et **items** les noms

C# : accès aux bases de données

■ Les espaces de nom à utiliser

- Selon le SGBD, un fournisseur d'accès est utilisé à travers un espace de nom
 - ✓ `System.Data.SqlClient` pour Microsoft SQL Server
 - ✓ `System.Data.OleDb` pour accéder à tout SGBD disposant de driver OleDb (Oracle – SQL Server – MySql – PostgreSQL ...)
 - ✓ `System.Data.OracleClient` pour le SGBD Oracle
 - ✓ `System.Data.ODBC` pour tout accès via le driver ODBC
- Les SGBD du marché offrent leur fournisseur d'accès :
 - ✓ `MySql.Data.MySqlClient` pour le SGBD MySql
 - ✓ `Oracle.DataAccess.Client` pour le SGBD Oracle

C# : accès aux bases de données

■ La connexion au SGBD

- Nécessite un objet **Connection** dont la classe est:
 - ✓ **SqlConnection** pour l'espace de nom System.Data.Client
 - ✓ **OleDbConnection** pour le fournisseur OleDb
 - ✓ **MySqlConnection** pour MySql.data.MySqlClient
 - ✓ **OracleConnection** pour Oracle.DataAccess.Client
- L'objet Connection nécessite une chaîne de connexion :
 - ✓ Pour **SqlConnection** la chaîne est :
 - *"Data Source = <serveur> ; Integrated Security = SSPI ; Initial Catalog = <labase>"* pour une authentification Windows
 - *"Data Source = <serveur> ; User Id = <nom_utilisateur> ; password = <mot de passe>; Initial Catalog = <labase>"*

C# : accès aux bases de données

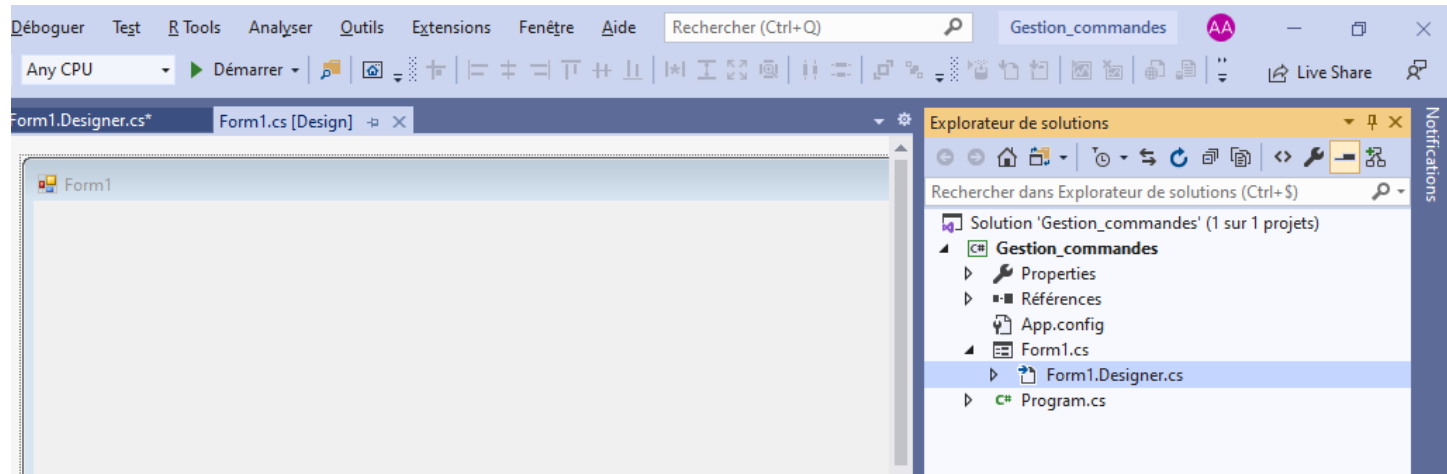
■ La connexion au SGBD

- L'objet **Connection** nécessite une chaîne de connexion :
 - ✓ Pour **OleDbConnection** la chaîne est :
 - *"Provider=MSDAORA; Data Source=<lien_base>;Persist Security Info = False; Integrated Security = False; User Id=<utilisateur>; Password=<mot_de_passe>"* pour Oracle
 - ✓ Pour **MySqlConnection** la chaîne est :
 - *"server=<machine>; database=<labase>;uid=<utilisateur>; pwd=<mot_de_passe>"*;
 - ✓ Pour **OracleConnection** la chaîne est :
 - *"Data Source = <lien_Base_donnee>; User id =<utilisateur>;Password = <mot_de_passe>"*
 - ✓ Exemple: *laconnexion = new MySqlConnection(lachaine);
laconnexion.open();* //laconnexion est supposée déclarée

C# : accès aux bases de données

■ Exemple connexion à Mysql

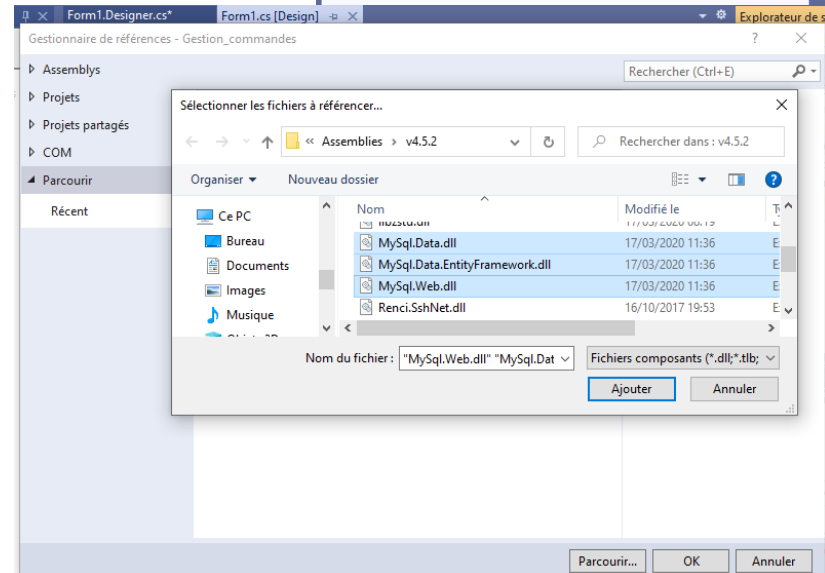
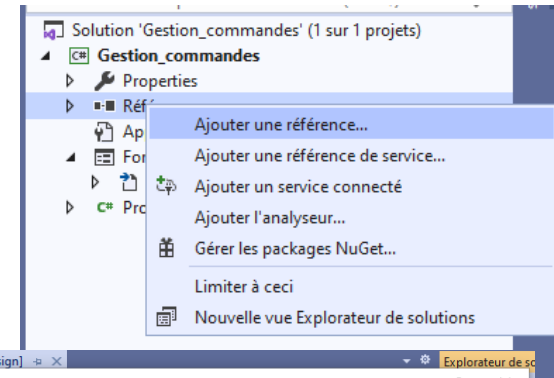
- Téléchargement du connecteur de MySQL pour .NET :
 - ✓ À l'adresse : <https://dev.mysql.com/downloads/connector/net/>
- Installation du connecteur :
 - ✓ lancer l'installation du fichier msi téléchargé et suivre les étapes
- Lancer Visual Studio et créer un projet Windows Form, Web ou console



C# : accès aux bases de données

■ Exemple connexion à Mysql

- Cliquez-droit sur Références dans Solution de projet
 - ✓ Puis **Ajouter une référence**
- Cliquez dans onglet Parcourir
 - ✓ Puis sur le bouton **Parcourir**



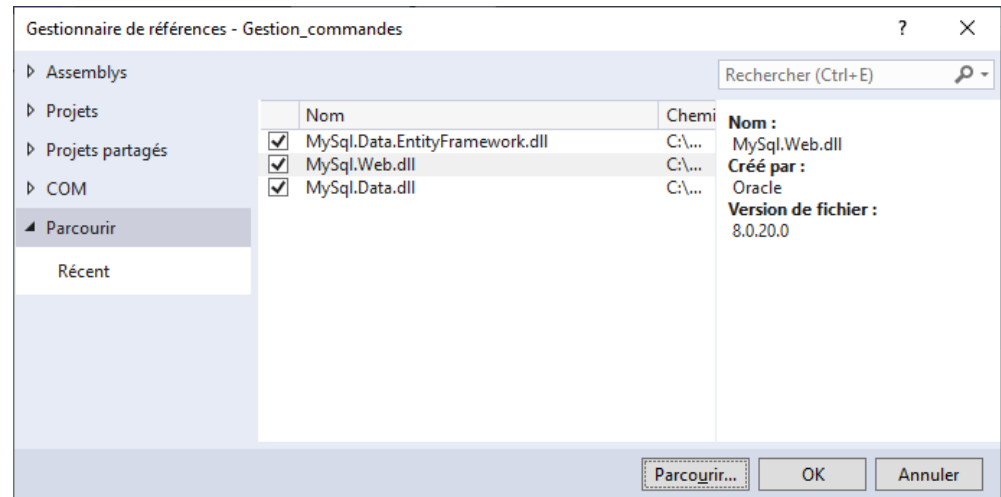
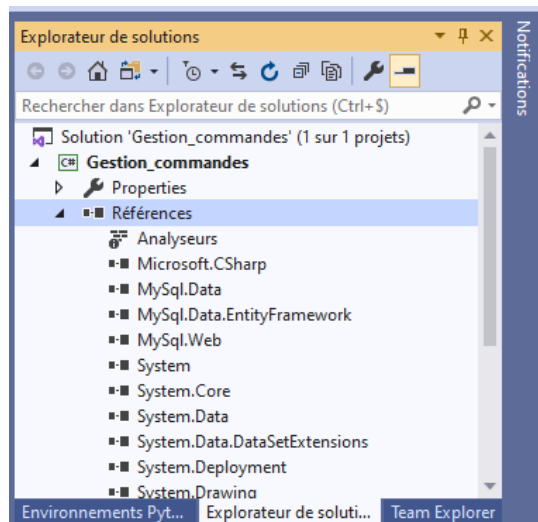
Dans le répertoire d'installation du connecteur, sélectionner les dll suivantes : **MySQL.Data.dll**, **MySQL.Data.EntityFramework.dll** et **MySQL.Web.dll**. Puis cliquer sur **Ajouter**

C# : accès aux bases de données

■ Exemple connexion à Mysql

✓ Cliquer ensuite sur **Ok**

Les espaces de noms apparaissent dans Références



On peut utiliser les objets des espaces de noms en tapant par exemple : `using MySql.Data;`
`using MySql.Data.MySqlClient;`

C# : accès aux bases de données

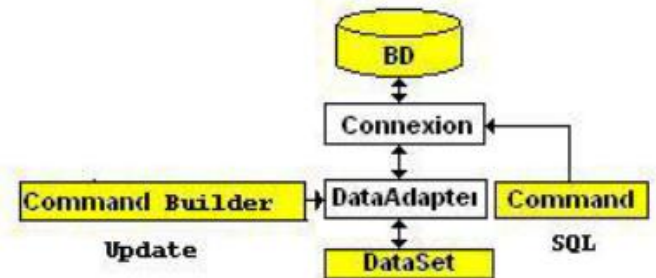
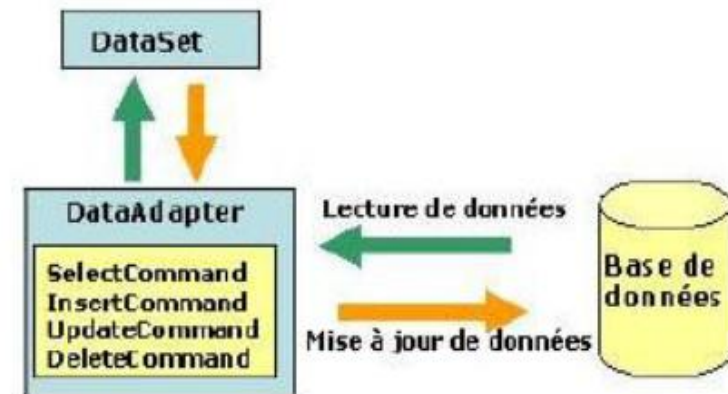
■ L'accès aux données : mode non connecté

- Utilise les objets suivants :
 - ✓ **DataSet** : représentation des données en mémoire, chargé à partir de la base de données
 - ✓ **DataAdapter** par sa méthode **Fill** charge le DataSet et par sa méthode **Update** met à jour la base de données. L'objet est :
 - **MySqlDataAdapter** pour `MySql.Data.MySqlClient`
 - **OracleDataAdapter** pour `Oracle.DataAccess.Client`
 - ✓ Le **DataAdapter** peut utiliser un objet **Command**
 - *laCommande = New OleDbCommand(larequete) ;*
leDataAdapter = New OleDbDataAdapter(laCommande) ;
//initialiser l'objet Command
laCommande.Connection() = laConnexion ;

C# : accès aux bases de données

L'accès aux données : mode non connecté

- Utilise les objets suivants :
 - ✓ Le **DataAdapter** utilise :
 - Soit les propriétés : **SelectCommand** – **InsertCommand** – **UpdateCommand** – **DeleteCommand** pour les requêtes paramétrées
 - Soit l'objet **CommandBuilder** qui modifie les valeurs changées dans le **DataAdapter**



C# : accès aux bases de données

■ L'accès aux données : mode non connecté

- Exemples : création et remplissage d'un DataSet

✓ // Création d'objets DataSet et DataAdapter

```
DataSet leDataset = New DataSet("Biblio" );
```

```
SqlDataAdapter leDataAdapter = New SqlDataAdapter("select  
* from Etudiant", laConnexion);
```

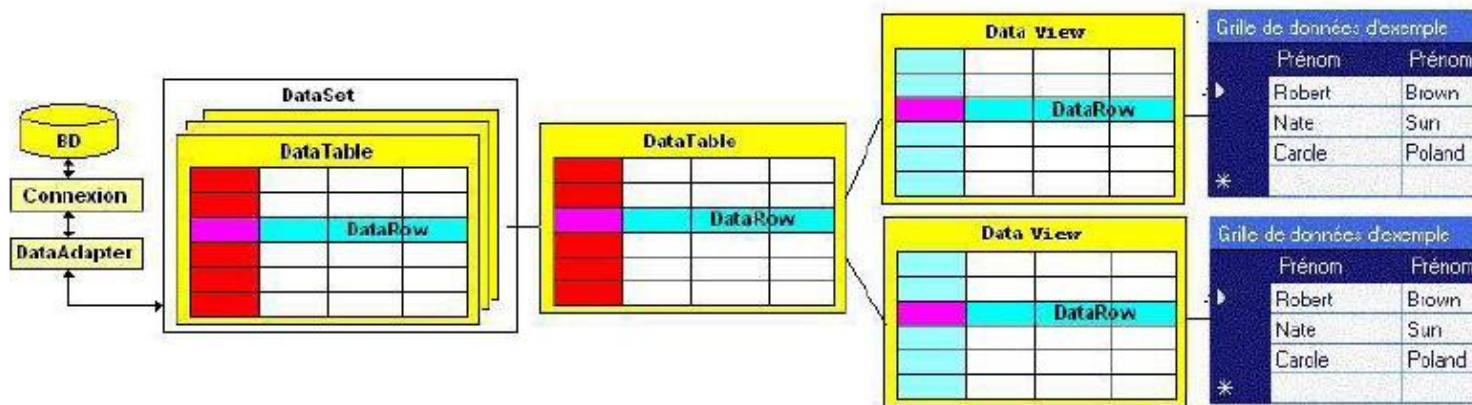
//remplissage du DataSet avec la table Etudiant

```
leDataAdapter.Fill(leDataset, "Etudiant");
```

C# : accès aux bases de données

L'accès aux données : mode non connecté

- Les objets du **DataSet** :
 - ✓ Le **DataTable** donne une représentation d'une table, il contient des **DataRow** qui représentent les enregistrements (lignes)
 - ✓ Le **DataView** représente une vue ou une table, créé à partir d'un **DataTable**, contient également des **DataRow**



C# : accès aux bases de données

L'accès aux données : mode non connecté

- Le DataTable

- ✓ Création d'un DataTable : propriété Tables du DataSet

- leDataAdapter.Fill(leDataSet, "Classe");
DataTable leDataTable1 = leDataSet.Tables["Classe"];

- ✓ insertion de données avec DataRow

- Création d'un enregistrement vierge avec méthode NewRow()

- DataRow leDataRow = leDataTable1.NewRow();

- Affectation de valeur aux différents champs et ajout de la ligne

- leDataRow["Code_cl"] = "Ing Info2"; ...
leDataTable1.Rows.Add(leDataRow);

- Modification de valeurs dans le DataAdapter et maj de la base de données

- laCommandBuilder = new SqlCommandBuilder(leDataAdapter);
leDataAdapter.Update(leDataSet, "Classe");

C# : accès aux bases de données

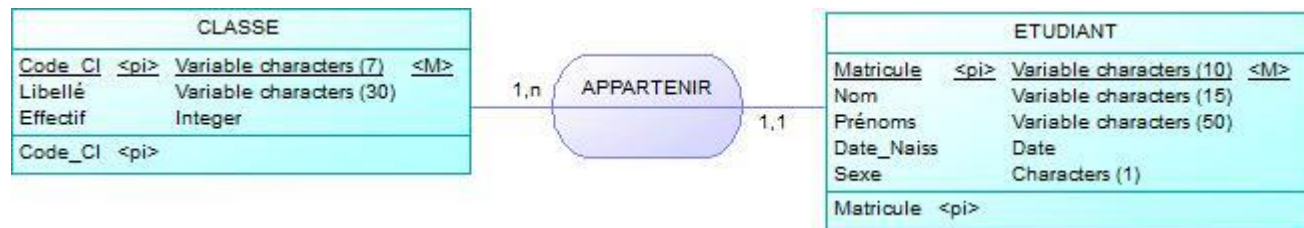
L'accès aux données : mode non connecté

- Le DataTable
 - ✓ Recherche de données avec méthode **Select** renvoyant tableau de DataRow
 - `lesDataRows = leDataTable1.Select(critère, ordre);`
 - critère : expression de critère SQL
 - ordre : nom de colonne suivi de ASC ou DESC
 - ✓ Récupération des valeurs d'une recherche : ligne i champ j
 - `textBox1.Text = lesDataRows[i][j].ToString();`
 - ✓ Utilisation de fonction d'agrégat avec méthode **Compute**
 - `leDataTable.Compute(expr, critère);`
 - expr : chaine contenant fonction de calcul
 - critère : expression de critère SQL
 - `float laSomme = leDataTable.Compute("Sum(Prix)", "Categorie=2,And Type='Bureautique')";`

C# : accès aux bases de données

L'accès aux données : mode non connecté

- Exemple d'accès avec MySql
 - ✓ Soit le diagramme suivant :



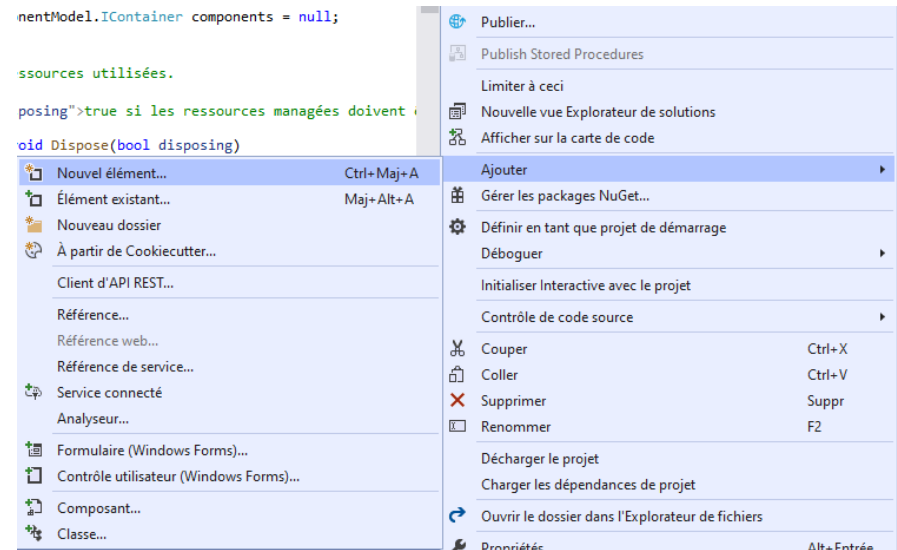
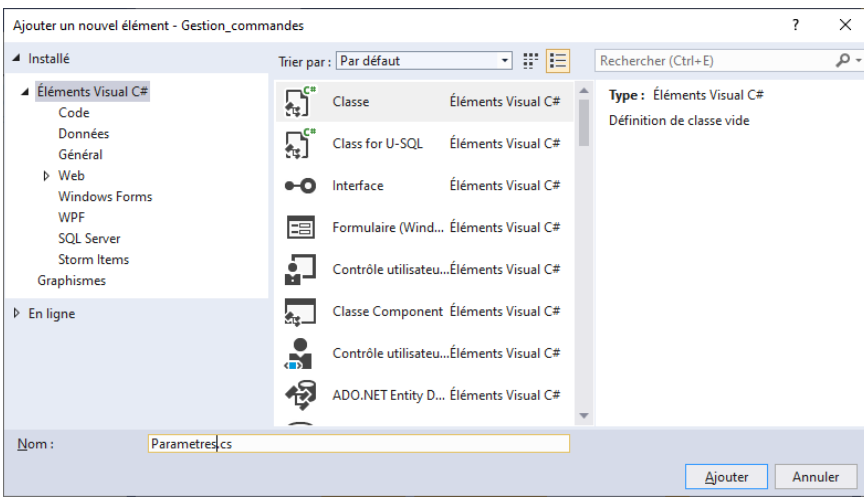
- ✓ Qui génère le schéma suivant :



C# : accès aux bases de données

L'accès aux données : mode non connecté

- Exemple d'accès avec MySql
 - ✓ Création des objets **Connection**, **DataSet**, **DataAdapter** et autres
 - On crée une classe statique pour déclarer ces objets
 - On clique-droit sur le nom du projet dans Explorateur de solution
 - Puis **Ajouter/Nouvel élément**
 - On donne un nom à la classe, puis **Ajouter**



C# : accès aux bases de données

L'accès aux données : mode non connecté

- Exemple d'accès avec MySQL
 - ✓ Création des objets Connection, DataSet, DataAdapter et autres
 - On crée une classe statique pour déclarer ces objets

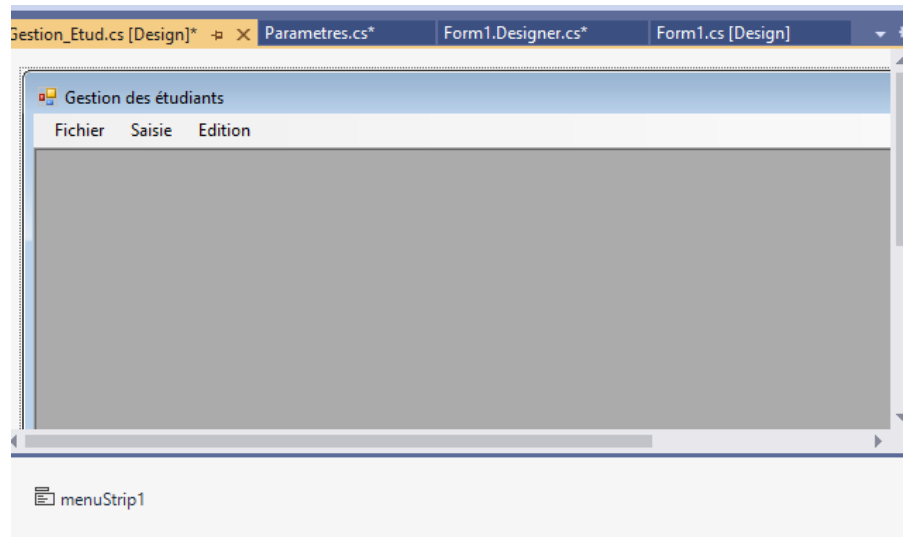
```
6 using System.Threading.Tasks;
7 using MySql.Data;
8 using MySql.Data.MySqlClient;
9
10 namespace Gestion_commandes
11 {
12     Or références
13     static class Parametres
14     {
15         public static MySqlConnection laconnexion;
16         public static DataSet leDataSet;
17         public static MySqlDataAdapter leDataAdapter;
18         public static MySqlCommandBuilder laCommandBuilder;
19     }
20 }
```

- ✓ Le programme va disposer d'une fenêtre principale et d'1 menu
 - Le menu **Fichier** avec sous-menu **Se Connecter/Se Déconnecter/Quitter**
 - Le menu **Saisie** avec sous-menu **Classe/Etudiant**
 - Le menu **Edition** avec sous-menu **les étudiants d'une classe**

C# : accès aux bases de données

L'accès aux données : mode non connecté

- Exemple d'accès avec MySql
 - ✓ Le programme va disposer d'une fenêtre principale et d'1 menu
 - On ajoute un nouvel élément et on choisit **Windows Form**
 - On ajoute un contrôle **MenuStrip** et on élabore le menu
 - On positionne la propriété **IsMdiContainer** du formulaire à **True**

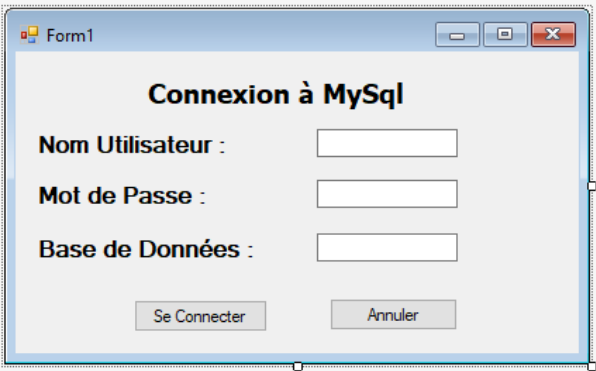


C# : accès aux bases de données

L'accès aux données : mode non connecté

- Exemple d'accès avec MySql
 - ✓ Élaboration du formulaire de connexion
 - On ajoute un formulaire qu'on nomme Connexion et on dispose les éléments sur le formulaire

Dans le click de Se Connecter on définit le code de connexion



The screenshot shows a standard Windows application window titled 'Form1'. Inside the window, the text 'Connexion à MySql' is centered at the top. Below this, there are three input fields arranged vertically. The first is labeled 'Nom Utilisateur :', the second 'Mot de Passe :', and the third 'Base de Données :'. Each label is followed by a rectangular text box. At the bottom of the form, there are two buttons: 'Se Connecter' on the left and 'Annuler' on the right. The window has standard Windows window controls (minimize, maximize, close) in the top right corner.

C# : accès aux bases de données

L'accès aux données : mode non connecté

- Exercice
 - ✓ Élaborer un formulaire de connexion au SGBD Oracle
 - ✓ Élaborer un formulaire de saisie de données dans la Table classe

C# : accès aux bases de données

L'accès aux données : mode non connecté

- Le **DataView**
 - ✓ On crée un **DataView** à l'aide de sa propriété **Table**
 - `leDataView.Table = leDataTable1;`
 - ✓ Tri et filtre des données d'un **DataView**
 - `leDataView.Sort = expr;`
expr : nom de colonne suivi de ASC ou DESC « Nom DESC »
 - `leDataView.RowFilter = critère;`
critère : expression de critère SQL
 - ✓ Recherche avec **Find** ou **FindRows**
 - On trie les données puis on effectue la recherche
`leDataView.Sort = "Matricule";`
`int i = leDataView.Find("ASSALE");`
■ `DataRow[] lesDataRows = leDataView.FindRows(critère);`

C# : accès aux bases de données

L'accès aux données : mode non connecté

- Principe des requêtes paramétrées avec les propriétés `xxxCommand` d'un `DataAdapter`
 1. Définition de commande avec la propriété `xxxCommand` du `DataAdapter`
 2. Spécification des paramètres en utilisant la méthode `Add` de l'objet `Parameters`
 3. Renseignement des valeurs des paramètres via un `DataTable` ou via la propriété `Value` de l'objet `Parameters`
 4. Mise à jour dans la base via la méthode `Update` du `DataAdapter`

C# : accès aux bases de données

L'accès aux données : mode non connecté

- Exemple de requêtes paramétrées avec `InsertCommand`
 - ✓ Définition de la commande
 - `String larequete_ins = "insert into classe values(@code_cl,@libelle,@effectif)";`
`leDataAdapter.InsertCommand = new SqlCommand(larequete_ins, laconnexion);`
 - `@code_cl`, `@libelle` et `@effectif` sont des paramètres, on aurait pu mettre ?
 - ✓ Spécification des paramètres
 - `leDataAdapter.InsertCommand.Parameters.Add("@code_cl", SqlDbType.NVarChar, 7, "Code_cl");`
`leDataAdapter.InsertCommand.Parameters.Add("@libelle", SqlDbType.NVarChar, 50, "libelle");`
`leDataAdapter.InsertCommand.Parameters.Add("@effectif", SqlDbType.Int, 7, "effectif");`

C# : accès aux bases de données

L'accès aux données : mode non connecté

- Exemple de requêtes paramétrées avec **InsertCommand**
 - ✓ Spécification des paramètres et ajout de valeurs via **Value**
 - `leDataAdapter.InsertCommand.Parameters.Add("@code_cl", SqlDbType.NVarChar, 7, "Code_cl").Value = "Ing Info1";`
`leDataAdapter.InsertCommand.Parameters.Add("@libelle", SqlDbType.NVarChar, 50, "libelle").Value = "Informatique 1ère année";`
`leDataAdapter.InsertCommand.Parameters.Add("@effectif", SqlDbType.Int, 7, "effectif").Value = 12;`
 - ✓ Renseignement des valeurs des paramètres via un **DataTable**
 - `leDataRow = leDataTable1.NewRow();`
`leDataRow["Code_cl"] = "Ing Info1";`
`leDataRow["libelle"] = "Informatique 1ère année";`
`leDataRow["Effectif"] = 12;`
`leDataTable1.Rows.Add(leDataRow);`

C# : accès aux bases de données

L'accès aux données : mode non connecté

- Exemple de requêtes paramétrées avec `UpdateCommand`
 - ✓ Définition de la commande
 - ```
String larequete_upd = "update set code_cl= @code_cl, libelle=@libelle, effectif=@effectif where code_cl=@code_cl1";
leDataAdapter.UpdateCommand = new SqlCommand(larequete_upd, laconnexion);
```
  - ✓ Spécification des paramètres
    - ```
SqlParameter leparam1 = leDataAdapter.UpdateCommand.  
Parameters.Add("@code_cl", SqlDbType.NVarChar,7,"Code_cl");  
SqlParameter leparam2 = leDataAdapter.UpdateCommand.  
Parameters.Add("@libelle", SqlDbType.NVarChar, 50, "libelle");  
SqlParameter leparam3 = leDataAdapter.UpdateCommand.  
Parameters.Add("@effectif", SqlDbType.Int, 7, "effectif");  
SqlParameter leparam4 = leDataAdapter.UpdateCommand.  
Parameters.Add("@code cl1", SqlDbType.NVarChar, 7, "Code cl");
```

C# : accès aux bases de données

L'accès aux données : mode non connecté

- Exemple de requêtes paramétrées avec `UpdateCommand`
 - ✓ Valorisation des paramètres via propriété `Value`
 - `leparam1.Value = "Ing Info3";`
`leparam2.Value = "Informatique 3ème année";`
`leparam3.Value = 15;`
`leparam4.Value = "Ing Info1";`
 - ✓ Valorisation des paramètres via un objet `DataTable`, on modifie les valeurs pour une ligne spécifique
 - `leDataTable1.Rows[numligne][0] = "Ing Info3";`
`leDataTable1.Rows[numligne][1] = "Informatique 3ème année";`
`leDataTable1.Rows[numligne][2] = 15;`
`leparam4.Value = "Ing Info1";`

C# : accès aux bases de données

L'accès aux données : mode non connecté

- Exemple de requêtes paramétrées avec `DeleteCommand`
 - ✓ Définition de la commande
 - `String larequete_del = "delete from classe where code_cl= @code_cl";
leDataAdapter.DeleteCommand = new SqlCommand(larequete_del,
laconnexion);`
 - ✓ Spécification des paramètres
 - `SqlParameter leparam = leDataAdapter.DeleteCommand.
Parameters.Add("@code_cl", SqlDbType.NVarChar,7,"Code_cl");`
 - ✓ Valorisation des paramètres
 - `leparam.Value = "Ing Info3"; // ou`
 - `leDataTable.Rows[numligne][0] = "Ing Info3"; // pour une ligne`

C# : accès aux bases de données

■ Travaux Pratiques N°1

- Soit la base de données où un Fournisseur(NumF, Nom, Adresse) passe plusieurs Commande(Num_cmde, date_cmde, Montant). Chaque commande comprend au moins un Produit(Code_prdt, Designation, Référence) avec la quantité commandée et le prix lors de la commande
 - ✓ Définir des formulaires pour gérer les Fournisseur – Commande et Produit
 - ✓ En mode non connecté avec objet CommandBuilder

C# : accès aux bases de données

L'accès aux données : mode connecté

- Utilise les objets suivants :
 - ✓ **Command** s'instancie avec une requête et une connexion et dispose des méthodes suivantes :
 - **ExecuteReader** retourne un objet **DataReader** résultat d'un Select
 - **ExecuteScalar** exécute une fonction ou procédure stockée
 - **ExecuteNonQuery** exécute une requête ne retournant pas de lignes
 - ✓ **DataReader** objet retourné lors de l'exécution d'une requête
 - **Read()** charge une ligne en mémoire et passe à la ligne suivante du jeu
 - **GetInt32()**, **GetDouble()**, **GetString()** pour récupérer les valeurs des colonnes dans leur type respectif
 - **Close()** ferme l'objet **DataReader**
 - **NextResult()** passe à un autre jeu d'enregistrements s'il y en a +sieurs

C# : accès aux bases de données

L'accès aux données : mode connecté

- Objet **Command** dispose de :
 - ✓ collection **Parameters** pour définir les paramètres d'une requête paramétrée ou procédure stockée
 - Méthode Add utilise 4 paramètres : nom du paramètre – type de données – taille de la colonne – nom de la colonne source
 - *SqlParameter leParametre = laCommande.Parameters.Add("@nom", SqlDbType.Char, 25, "Nom");*
 - Propriété **Value** permet de fixer la valeur du paramètre
 - Propriété **Direction** permet pour les procédures stockées de préciser si le paramètre est en entrée (**Input**), en sortie (**Output**), de retour (**ReturnValue**)
 - ✓ Propriété **CommandType** prend pour valeur **CommandType.Text** si c'est une instruction SQL ou **CommandType.StoredProcedure** pour procédure stockée

C# : accès aux bases de données

L'accès aux données : mode connecté

- Exemple d'exécution de commande **Select** :
 - ✓ Définition de la requête :
 - *String larequete = "select * from Classe";*
laCommande = new SqlCommand(larequete, laconnexion);
 - ✓ Exécution et récupération des valeurs de la requête
 - *SqlDataReader leDataReader= laCommande.ExecuteReader();*
while (leDataReader.Read())
{
comboBox2.Items.Add(leDataReader.GetString(0));
}

C# : accès aux bases de données

L'accès aux données : mode connecté

- Exemple d'exécution de commande **Insert** :
 - ✓ Définition de la requête :
 - *String larequete = "insert into classe values('" + lecode_cl + "'," + lelibelle + "'," + leffectif + "');" ;*
laCommande = new SqlCommand(larequete, laconnexion);
 - ✓ Exécution de la requête
 - *laCommande.ExecuteNonQuery();*

C# : accès aux bases de données

L'accès aux données : mode connecté

- Exemple d'exécution de fonction stockée :

- ✓ Définition de la requête :

- *String larequete = "nombre_emprunt";*
laCommande = new SqlCommand(larequete, laconnexion);
laCommande.CommandType = CommandType.StoredProcedure;

- ✓ Spécification des paramètres et exécution de la commande

- *SqlParameter lepara1 = laCommande.Parameters.Add("@lenom",*
SqlDbType.NVarChar, 20, "Nom"); lepara1.Value = lenom; SqlParameter
lepara2 = laCommande.Parameters.Add("@leprenom",
SqlDbType.NVarChar, 50, "prenoms");
lepara2.Value = leprenom; SqlParameter lepara3 =
laCommande.Parameters.Add("ReturnValue", SqlDbType.Int);
lepara3.Direction = ParameterDirection.ReturnValue;
laCommande.ExecuteScalar(); Nombre.Text = lepara3.Value.ToString();

C# : accès aux bases de données

Travaux Pratiques N°2

- Reprise du Travail Pratique 1
 - ✓ En mode connecté
 - ✓ Définir une fonction qui retourne le nombre de commande d'un fournisseur au niveau de la base de données et appeler cette fonction à travers un formulaire